ACTL3142 Tutorial

Week 7: Cross-Validation & Regularisation

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T2 2026

Recap

From Week 5 (Generalised Linear Models):

- Three components: distribution (EDF), linear predictor $\eta = \mathbf{x}'\boldsymbol{\beta}$, link function $g(\mu) = \eta$
- EDF form: $f(y;\theta,\psi) = \exp\left[\frac{y\theta - b(\theta)}{\psi} + c(y;\psi)\right]$
- Deviance for model comparison, pseudo-R^2, residual diagnostics

New problem: how do we know our model will perform well on **unseen data**? And can we prevent overfitting by constraining the model?

Today: **Cross-validation** (honest error estimation) and **Regularisation** (Ridge & Lasso).

Ultimate Goal: Out-of-Sample Performance

Want to build a model that predicts well on **new, unseen data**. Training error alone is misleading, complex models are designed to fit the training data well.

Indirect approaches (what we've used so far):

- AIC, BIC, Mallow's C_p, adjusted R^2, etc.
- These metrics adjust training error for model complexity to *approximate* out-of-sample (OOS) error, without ever holding out new data.

Direct approach (this week):

- Actually **hold out data** the model has never seen
- Run the model on it and measure prediction error directly

Both approaches are valid. The direct approach gives a real OOS error estimate, while indirect metrics are convenient when data is scarce.

Train / Validation / Test Split

To use the direct approach properly, we split data into **three** parts. A common setup is **60 / 20 / 20**:

1. **Training set** (60%): fit your candidate models here
2. **Validation set** (20%): evaluate each candidate on this held-out data. Build many models, pick the one with the **lowest validation error**.
3. **Test set** (20%): touch this **only once at the very end** to report a final OOS performance metric

Why not just train/test?

If you use the test set to choose between models, it's no longer "unseen". You're implicitly fitting to it. The validation set absorbs this model-selection step so the test set stays clean for final reporting.

Test Error vs Test *Set* Error

Two terms that are easy to conflate:

- **Test error**: a model's true expected error on unseen data (a theoretical quantity, never observed directly)
- **Test set error**: the error measured on the held-out test set, which **estimates** the test error

For a single model fixed in advance, the validation error is an (almost) unbiased estimate of its test error. The complication comes from **selection**.

Selection makes the validation error optimistic

Choosing the model with the lowest validation error uses the validation set for optimisation, not just evaluation. Among models of similar true quality, the one that scores best is partly winning on noise, so its validation error **understates** its true test error.

The test set takes no part in selection, so it gives an honest estimate of the chosen model. A test error far above the validation error signals that selection has overfit the validation set.

From Validation Sets to Cross-Validation

The validation set approach is simple, but even with a well-shuffled split:

- High variance: a different split gives a different answer, and shuffling only randomises *which* single split you get
- Wasteful: the model trains on only 60% of the data

Cross-validation fixes both: every observation is used for validation exactly once, so it **averages over many splits** (lower variance) and **reuses all the data** (less waste). It replaces the validation split; you still hold out a separate test set for the final estimate.

Two main uses of CV:

- **Hyperparameter tuning**: e.g. choosing λ in ridge/lasso, or a polynomial degree. Build many models, pick the lowest CV error.
- **Model comparison**: e.g. OLS vs ridge vs lasso, as long as the same loss makes the errors comparable.

k-Fold Cross-Validation

Procedure:

1. Randomly split the data into k roughly equal-sized **folds**
2. For each fold $j = 1, \ldots, k$:
   - Hold out fold j as the validation set
   - Train the model on the remaining $k - 1$ folds
   - Compute the prediction error on fold j
3. Average the k error estimates:

$$\text{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^{k} \text{MSE}_j$$

Every observation is used for validation exactly once.

LOOCV and Choice of k

LOOCV (Leave-One-Out CV): set $k = n$. Each fold has one observation.

- Nearly unbiased (training set is $n - 1$ observations, almost the full dataset)
- Expensive: fit the model n times
- For linear models, a shortcut exists: $\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2$

Bias-variance tradeoff in choosing k:

	Bias of CV estimate	**Variance of CV estimate**
Small k (e.g. 5)	Higher	Lower
Large k (e.g. n)	Lower	Higher

The next slide explains why each direction moves the way it does.

Choosing k: Bias and Variance of the CV *Estimate*

Careful: this tradeoff is about the bias and variance of the CV **estimate of test error**, not the bias and variance of the model itself.

Why larger k gives lower bias:

- CV aims to estimate the error of a model trained on the **full** data
- Each fold trains on only part of it, and less data means a worse model, so CV **overestimates** the error (a pessimistic bias)
- Larger $k \Rightarrow$ bigger training folds $\Rightarrow$ closer to the full data $\Rightarrow$ less bias

Why larger k gives higher variance:

- At $k = n$ (LOOCV) the training sets are nearly identical, so the fitted models and their fold errors are **highly correlated**
- Averaging highly correlated numbers barely cancels noise, so the estimate itself is more variable

So $k = 5$ or 10 is the sweet spot: almost as unbiased as LOOCV, with lower variance.

Q1: k-Fold Cross-Validation

★ Solution (ISLR2, Q5.3) Consider k-fold cross-validation.

1. a. Explain how k-fold cross-validation is implemented.

   b. What are the advantages and disadvantages of k-fold cross-validation relative to:

      - The validation set approach?
      - LOOCV?

Q1: Walkthrough

(a) How k-fold CV is implemented:

1. Split data randomly into k equal folds
2. For each fold: hold it out, train on the rest, predict on held-out fold
3. Average the k fold errors (this estimates the test error)

(b) Compared to the validation set approach:

+ Lower variance (averages over k splits, not just one)
+ Uses data more efficiently (every point is used for validation once)
– More expensive (fit k models instead of 1)

Compared to LOOCV:

+ Cheaper (k fits vs n fits)
+ Lower variance (training sets overlap less, so fold errors are less correlated)
– Slightly higher bias (each training set is smaller than $n - 1$)

Ridge Regression

OLS minimises RSS $= \sum(y_i - \mathbf{x}_i'\boldsymbol{\beta})^2$. Ridge adds an L_2 penalty:

$$\hat{\boldsymbol{\beta}}^{\text{ridge}} = \arg\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$$

- $\lambda \geq 0$ is the **tuning parameter** (a hyperparameter chosen by CV)
- $\lambda = 0$: no penalty, recover OLS
- $\lambda \to \infty$: all $\hat{\beta}_j \to 0$ (only intercept survives)
- **Shrinks** coefficients toward zero but keeps all predictors (no selection)

Closed-form: $\hat{\boldsymbol{\beta}}^{\text{ridge}} = (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}'\mathbf{y}$

Why it helps: when p is large or predictors are correlated, OLS has high variance. The $\lambda\mathbf{I}$ term stabilises the inversion, trading a small bias increase for a large variance decrease.

Important: standardise predictors first, since the penalty treats all β_j equally.

Lasso Regression

Replace the L_2 penalty with an L_1 penalty:

$$\hat{\boldsymbol{\beta}}^{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i' \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right\}$$

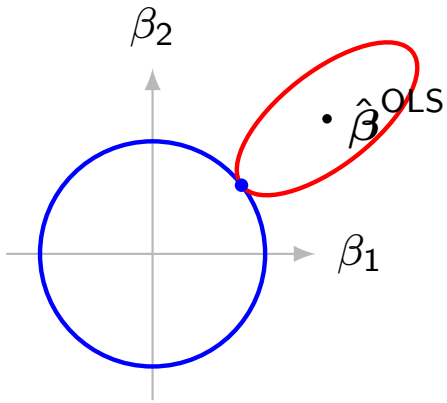
- The L_1 penalty can shrink coefficients to **exactly zero**
- This performs **variable selection**: irrelevant predictors get dropped
- No closed-form solution (need numerical optimisation)

Equivalent "budget" formulation:

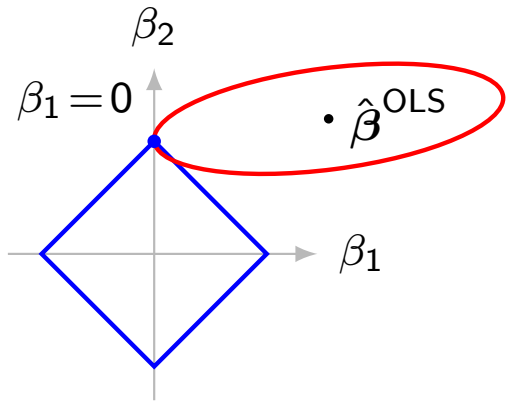
$$\min_{\boldsymbol{\beta}} \text{ RSS} \quad \text{subject to} \quad \sum_{j=1}^{p} |\beta_j| \leq s$$

Why Does Lasso Give Sparse Solutions?

Both minimise RSS subject to a size limit on $\boldsymbol{\beta}$. RSS contours are **ellipses** centred at $\hat{\boldsymbol{\beta}}^{\text{OLS}}$; the solution is the **first point** an expanding contour touches the constraint region (the feasible point of lowest RSS).



Ridge: contact *off* the axes



Lasso: contact at a corner

The circle is **smooth**, so the contact sits on an axis only for a knife-edge (measure-zero) coincidence: in practice both $\beta_j \neq 0$.

The diamond's **corners lie on the axes** and jut out, so a whole range of ellipse positions hit a corner first, giving $\beta_j = 0$ exactly.

A ridge zero is not impossible, just practically never (nothing pulls the tangency onto an axis). Lasso's corner sits on the axis, so it gives zeros over a whole range of λ: genuine selection.

Ridge vs Lasso

	Ridge (L_2)	**Lasso (L_1)**
Penalty	$\lambda \sum \beta_j^2$	$\lambda \sum \lvert\beta_j\rvert$
Coefficients	Shrunk toward 0	Shrunk, some exactly 0
Variable selection	No	Yes
Correlated vars	Equal coefficients	May pick one, drop others
Solution	Closed-form	Numerical

Why bother? Both trade a small increase in bias for a larger decrease in variance. When OLS overfits (many predictors, collinearity), regularisation can improve test error.

Choosing λ: use cross-validation. In R: `cv.glmnet()` returns the λ that minimises CV error.

Both extend to any GLM (the penalty is added to the deviance, not the RSS). `glmnet`'s `alpha` blends the two: $\alpha = 1$ lasso, $\alpha = 0$ ridge, in between is the elastic net.

Q2: Lasso, Ridge, and Non-linear vs Least Squares

⋆ Solution (ISLR2, Q6.2) For parts (a) through (c), indicate which of i. through iv. is correct. Justify your answer.

2. a. The lasso, relative to least squares, is:

   i. More flexible and hence will give improved prediction accuracy when its increase in bias is less than its decrease in variance.

   ii. More flexible and hence will give improved prediction accuracy when its increase in variance is less than its decrease in bias.

   iii. Less flexible and hence will give improved prediction accuracy when its increase in bias is less than its decrease in variance.

   iv. Less flexible and hence will give improved prediction accuracy when its increase in variance is less than its decrease in bias.

   b. Repeat (a) for ridge regression relative to least squares.

   c. Repeat (a) for non-linear methods relative to least squares.

Q2: Walkthrough

The key idea: **less flexible** = more bias, less variance. **More flexible** = less bias, more variance. A method improves on OLS when the variance reduction outweighs the bias increase (or vice versa).

(a) Lasso vs least squares: (iii)

- Lasso is **less flexible** (some coefficients shrunk to zero)
- Trades a small bias increase for a larger variance decrease

(b) Ridge vs least squares: (iii)

- Same reasoning: ridge restricts coefficient sizes, so less flexible than OLS

(c) Non-linear methods vs least squares: (ii)

- Non-linear methods are **more flexible** (can capture curves, interactions)
- Trades a variance increase for a larger bias decrease

Q3: Lasso Budget Constraint Behaviour

⋆ Solution (ISLR2, Q6.3) Suppose we estimate the regression coefficients in a linear regression model by minimising

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_i x_{ij}\right)^2 \text{ subject to } \sum_{j=1}^{p}|\beta_j| \leq s$$

for a particular value of s. For parts (a) through (e), indicate which of i. through v. is the (only) correct choice. Justify your answer.

3. a. As we increase s from 0, the training RSS will:

    i. Increase initially, and then eventually start decreasing in an inverted U shape.

    ii. Decrease initially, and then eventually start increasing in a U shape.

    iii. Steadily increase.

    iv. Steadily decrease.

    v. Remain constant.

   b. Repeat (a) for test RSS.

   c. Repeat (a) for variance of parameters' estimates.

   d. Repeat (a) for (squared) bias of parameters' estimates.

   e. Repeat (a) for the irreducible error.

Q3: Walkthrough

Minimise RSS subject to $\sum |\beta_j| \leq s$. As s increases from 0, the constraint relaxes and the model becomes **more flexible** (at $s = \infty$ we recover OLS).

(a) Training RSS:	**(iv)** Steadily decrease	(better fit)
(b) Test RSS:	**(ii)** U-shape	(then overfits)
(c) Variance:	**(iii)** Steadily increase	(more flexible)
(d) Bias^2:	**(iv)** Steadily decrease	(less rigid)
(e) Irred. error:	**(v)** Constant	(model-free)

The "sweet spot" for test error is where bias-variance balance is optimal.

Applied Q4: College Data (OLS vs Ridge vs Lasso)

⋆ Solution (ISLR2, Q6.9, modified) Refer to the data set `College` from package `ISLR2`. In this exercise, we will predict the graduation rate `Grad.Rate` (last variable in the data set) using all the other variables as predictors.

4. a. Split the data set into a training set (2/3 of the data) and a test set (1/3).

   b. Fit a linear model using least squares on the training set, and report both the "training error" and "test error" obtained.

   c. Fit a ridge regression model on the training set. Find the MSE on the test set with the arbitrary choice $\lambda = 10$. Then, use function `cv.glm` to chose the "best" λ by cross-validation. Report the new test MSE obtained, and briefly discuss the result.

   d. Fit a lasso model on the training set, with λ chosen by cross-validation. Report the test MSE obtained, along with the number of non-zero coefficient estimates.

   e. Comment on the results obtained. How accurately can we predict `Grad.Rate`? Is there much difference among the test errors resulting from these three approaches (Linear Regression, Ridge and Lasso Regressions)?

Applied Q4: Hints

Pull this question up on your side. Key pointers for each part:

(a) Use `sample()` to split 2/3 train, 1/3 test

(b) Fit `lm(Grad.Rate ~ ., data = train)`, report training and test MSE

(c) Ridge: `glmnet(..., alpha = 0)`. First try $\lambda = 10$, then use `cv.glmnet()` to find the best λ. Compare test MSEs.

(d) Lasso: `glmnet(..., alpha = 1)` with CV-chosen λ. Report test MSE and count non-zero coefficients.

(e) Compare the three test MSEs. Typically all are similar here, but lasso gives a sparser model (fewer predictors).

Thanks for coming! See you next week.